

Práctica 4 SIW - Decisiones relevantes

Esta práctica consistía en implementar un script que calculara distintas medidas de similitud:

- Coeficiente de Dice
- Coeficiente de Jaccard
- Coseno
- Coeficiente de solapamiento

En este documento describo algunas de las decisiones que tomé para el desarrollo de esta práctica.

Primeramente, me basé en el pseudocódigo adjunto en el enunciado para definir las funciones `main` y `string_to_bag_of_words`. En esta segunda, tomé la decisión de utilizar **TextBlob** para realizar las distintas acciones sobre los textos (tanto los documentos de `cranfield` como las queries), siguiendo las recomendaciones de clase:

- Tokenización mediante `words` de `TextBlob`
- Paso de tokens a minúsculas
- Eliminación de *stop words* utilizando la colección de `nltk.corpus`
- Lematización mediante la función `lemmatize` de `TextBlob`
- Almacenamiento de la frecuencia de aparición junto a cada palabra

```
def string_to_bag_of_words(text):
    bow = {}
    zen = TextBlob(text)
    tokens = zen.words # tokenizar
    for token in tokens:
        token = token.lower() # minúsculas
        if token in stops: continue # ignorar si es una stopword
        token = Word(token).lemmatize() # lema
        bow[token] = bow.get(token, 0) + 1 # se almacena la frecuencia de
aparición
    return bow
```

Para encontrar el mejor texto asociado a cada query, utilicé una función `find_best_text` que, dependiendo de la medida de similitud deseada, llama a una función u otra. También implementé en el código un parámetro `boolean_search` el cual se puede configurar para que realice una búsqueda booleana o una búsqueda basada en frecuencias de aparición (el *bonus* indicado en la práctica). Así, se calcula por cada documento la puntuación con respecto a la consulta y se devuelve la mejor puntuación encontrada.

Las funciones de cálculo de medidas de similitud tienen la siguiente forma:

```
def dice_coefficient(q_bow, doc_bow, boolean_search):
    # aplicación booleana
    if boolean_search:
        set1, set2 = set(q_bow.keys()), set(doc_bow.keys())
        intersection = len(set1 & set2)
        return (2 * intersection) / (len(set1) + len(set2))

    # aplicación por frecuencias
    intersection_sum = 0
    sum_q = sum(q_bow.values())
    sum_d = sum(doc_bow.values())
    for term in q_bow:
        if term in doc_bow:
            intersection_sum += min(q_bow[term], doc_bow[term])

    denominator = sum_q + sum_d
    return (2 * intersection_sum) / denominator if denominator > 0 else 0.0
```

Cada medida se implementa según su fórmula específica.

Por último, diseñé una función `load_lines` que sirve para cargar tanto las queries como los documentos de cranfield, ya que implementa una expresión regular que funciona para ambos archivos:

```
pattern = r'^(I\s*\d+)\s+(.*)$'
```

Esta función, además, transforma automáticamente los textos en un diccionario que contiene el identificador (ej. I1) y una *bolsa de palabras* asociada.

```
for line in lines:
    match = re.match(pattern, line)
    if match:
        identifier, text = match.groups()
        dictionary[identifier] = string_to_bag_of_words(text)
```

Los resultados se cargan en un archivo `results.txt` donde se especifica el tipo de búsqueda utilizada (booleana o no) y los mejores textos y puntuaciones para cada consulta.

```
results.txt

SEARCH TYPE: BOOLEAN
QUERY: I 001
DICE: I878 (0.14814814814814814 score)
JACCARD: I878 (0.08 score)
COSINE: I878 (0.18392019594868114 score)
OVERLAPPING: I486 (0.45454545454545453 score)
```

QUERY: I 002

...

...